

Parallelization of an ACO Solver for the Constrained Vehicle Routing Problem

Francesco Zompanti (2012601)

Iacopo Tomini (2060878)

Abstract

This paper presents the design and analysis of a solver for the Constrained Vehicle Routing Problem based on Ant Colony Optimization. The report is organized around the mathematical formulation of the problem, the structure of the sequential baseline, and the analysis of the parallel implementations developed in the project. Particular attention is devoted to the algorithmic components that dominate execution time, such as probabilistic route construction, solution evaluation, and pheromone management.

1 Introduction

This report presents our project on the **Constrained Vehicle Routing Problem** solved through an **Ant Colony Optimization** approach. The external report used as a reference only provides the high-level organization of the document; the underlying problem, algorithm, data structures, and implementation choices are entirely different in our case.

1.1 Problem formulation

We consider a constrained vehicle routing problem on a complete graph $G = (V, E)$, where $V = \{0, 1, \dots, n\}$ is the set of nodes and node 0 denotes the depot. The set of customers is therefore $C = \{1, \dots, n\}$. Each arc $(i, j) \in E$ is associated with a travel cost $c_{ij} \geq 0$, derived in our instances from the Euclidean distance between nodes. A fleet of at most K vehicles is available, and each vehicle has capacity Q . In the instances handled by our solver, customer demands are unitary, that is, $q_i = 1$ for every $i \in C$, while $q_0 = 0$.

A feasible solution is a set of depot-to-depot routes such that each customer is visited exactly once and the total demand served by each route does not exceed the vehicle capacity. Let x_{ij}^k be a binary decision variable equal to 1 if vehicle k traverses arc (i, j) and 0 otherwise. A compact mathematical formulation can be written as follows:

$$\min \sum_{k=1}^K \sum_{i \in V} \sum_{j \in V, j \neq i} c_{ij} x_{ij}^k$$

subject to

$$\sum_{k=1}^K \sum_{j \in V, j \neq i} x_{ij}^k = 1 \quad \forall i \in C$$

$$\sum_{j \in C} x_{0j}^k \leq 1, \quad \sum_{i \in C} x_{i0}^k \leq 1 \quad \forall k = 1, \dots, K$$

$$\sum_{j \in V, j \neq i} x_{ij}^k - \sum_{j \in V, j \neq i} x_{ji}^k = 0 \quad \forall i \in V, \forall k = 1, \dots, K$$

$$\sum_{i \in C} q_i \sum_{j \in V, j \neq i} x_{ij}^k \leq Q \quad \forall k = 1, \dots, K$$

with

$$x_{ij}^k \in \{0, 1\} \quad \forall i, j \in V, i \neq j, \forall k = 1, \dots, K.$$

The objective minimizes the total travel cost of all routes. The first constraint ensures that every customer is served exactly once. The second enforces that each vehicle can leave from and return to the depot at most once. The third is the standard flow-conservation constraint, while the fourth encodes the vehicle-capacity limit. In the specific setting of our project, since all customer demands are equal to one, the capacity constraint can also be interpreted as a bound on the maximum number of customers assigned to each route.

1.2 Sequential baseline analysis

The sequential baseline implemented in the uploaded code follows a standard ACO structure with two practical additions: bounded pheromone updates and a repair-oriented route construction phase that preserves feasibility when some customers remain unassigned after the first pass.

Algorithm 1 Sequential baseline in `aco_sequential.c`

```
1: Input:  $n, K, Q, m, c, (\alpha, \beta, \rho, \tau_0)$ , seed
2: initialize pheromone matrix  $\tau$  and heuristic data
3: if  $m \leq 0$  then
4:    $m \leftarrow \text{CHOOSEAUTOTOTALANTS}(n)$ 
5: end if
6:  $S^* \leftarrow \emptyset; f^* \leftarrow +\infty$ 
7:  $\tau_{\max} \leftarrow \tau_0; \tau_{\min} \leftarrow 0.05\tau_0$ 
8: for  $t \leftarrow 0, 1, 2, \dots$  until timeout or stagnation do
9:   update candidate scores from  $\tau$ 
10:   $S^{\text{iter}} \leftarrow \emptyset; f^{\text{iter}} \leftarrow +\infty$ 
11:  for  $a \leftarrow 1$  to  $m$  do
12:    build and evaluate one ant solution
13:    if  $f_a < f^{\text{iter}}$  then
14:       $S^{\text{iter}} \leftarrow S_a; f^{\text{iter}} \leftarrow f_a$ 
15:    end if
16:  end for
17:  if  $f^{\text{iter}}$  improves  $f^*$  then
18:    update  $S^*, f^*$ , and pheromone bounds
19:    reset stagnation counters
20:  else
21:    increment stagnation counters
22:  end if
23:  evaporate pheromone on all arcs
24:  reinforce pheromone with  $S^{\text{iter}}$  and  $S^*$ 
25:  clamp  $\tau$  to  $[\tau_{\min}, \tau_{\max}]$ 
26:  if short-term stagnation is reached then
27:    mix  $\tau$  with the initial level  $\tau_0$ 
28:  end if
29: end for
30: return  $S^*, f^*$ 
```

Algorithm 2 Simplified feasible ant construction

```
1: function BUILDANTSOLUTION( $K, Q, c, \tau$ )
2:   initialize empty solution, visited set, and remaining
   customers  $r \leftarrow n$ 
3:   for  $k \leftarrow 1$  to  $K$  do
4:     start route  $R_k$  at depot 0 with load  $\ell_k \leftarrow 0$ 
5:     while  $r > (K - k)Q$  and  $\ell_k < Q$  do
6:       choose the next customer by the ACO rule
7:       if the choice is invalid or already visited then
8:         replace it with the nearest unvisited cus-
        tomer
9:       end if
10:      if no feasible customer exists then
11:        break
12:      end if
13:      append the customer to  $R_k$ , mark it visited,
      and decrement  $r$ 
14:    end while
15:    close  $R_k$  by returning to depot 0
16:  end for
17:  if  $r > 0$  then
18:    reopen routes from  $K$  down to 1
19:    greedily insert nearest unvisited customers while
    capacity remains
20:    close the modified routes again at the depot
21:  end if
22:  return the constructed solution
23: end function
```

In implementation terms, this pseudo-code is realized through four main components. The instance parser reads TSPLIB-style EUC_2D CVRP inputs, extracts the number of vehicles and the route ca-

capacity, validates the unit-demand assumption, and builds the complete distance matrix. The matrix and solution modules provide the aligned dense storage and the route-based solution representation. The ACO support routines manage random-number generation, candidate scoring, candidate selection, and route-construction data. Finally, the optimization loop coordinates ant generation, solution evaluation, best-solution tracking, pheromone bounding, and stagnation recovery.

This structured description is especially useful for the rest of the report, because it isolates the computational kernels that dominate execution time and therefore become the natural targets for optimization and parallelization: ant solution construction, probabilistic next-customer selection, route-cost evaluation, and pheromone management.

1.3 Problem and algorithmic challenges

The project combines combinatorial optimization and constrained route construction, which introduces several challenges:

- the search space grows rapidly with the number of customers;
- the constructive phase must preserve feasibility while still exploring competitive solutions;
- the algorithm repeatedly evaluates many probabilistic choices and route costs, making performance strongly dependent on memory access patterns and implementation efficiency.

2 OpenMP–MPI Roadmap

The OpenMP–MPI backend evolved through successive design iterations. The initial hybrid baseline established a functional starting point, but its scaling behavior exposed several architectural bottlenecks. A later iteration redesigned the solver according to high-performance computing principles, targeting cache efficiency, reduced synchronization, and communication patterns that remain sustainable on large instances. This section summarizes that evolution and explains why a subsequent attempt at *intra-ant parallelism* on CPUs failed despite its apparent theoretical appeal.

2.1 Initial iteration: baseline limitations

The initial hybrid implementation already exploited the natural mapping of ACO to distributed-memory execution: each MPI rank acted as a sub-colony, while OpenMP threads constructed local ant solutions. In practice, however, four bottlenecks limited both strong and weak scaling.

- **False sharing.** Telemetry and per-thread statistics were stored in non-aligned arrays of structures. Frequent writes from different threads caused repeated cache-line invalidations in L1/L2, reducing effective core throughput.
- **Fork-join overhead.** The OpenMP parallel region was entered and destroyed once per epoch. This imposed a fixed latency at every iteration, which became visible especially on medium-size instances with short epochs.
- **Serialized pheromone deposit.** The deposit phase was executed inside a `#pragma omp single` region, introducing a strict serial tail at the end of each epoch and therefore an Amdahl-style limit on speedup.
- **High MPI latency.** Pheromone synchronization was performed through one `MPI_Iallreduce` per row of the matrix, that is, $N + 1$ collectives per epoch. The resulting stream of small messages saturated the communication layer long before the network bandwidth itself became the limiting resource.

These issues meant that the baseline solver was parallel in structure but not yet HPC-ready in execution behavior.

2.2 Version 2: Architectural pillars

Version 2 restructured the OpenMP-MPI backend around four complementary ideas.

2.2.1 Persistent threading

The OpenMP region is created once outside the epoch loop and kept alive for the full run. Threads synchronize through lightweight barriers and shared state transitions instead of repeated fork-join cycles. This removes a fixed source of overhead and improves temporal locality in thread-private workspaces.

2.2.2 Cache awareness and adaptive auto-tuning

The second pillar is an explicitly cache-oriented data layout:

- all thread-private structures are padded and aligned to 64 bytes, matching the cache-line size of the target CPUs;
- hot buffers are organized to avoid inter-thread interference;
- the parameter K_{cand} is auto-tuned at startup from the detected L3 capacity.

The tuning rule was refined to account for both candidate indices and scores, for an effective footprint of 8 bytes per element with a 70% safety factor. The goal is to keep the hottest exploration struc-

tures resident in L3 and therefore reduce fallback traffic toward DRAM.

2.2.3 Fully parallel pheromone update

The serialized deposit phase of Version 1 was replaced by a fully parallel update based on hardware atomics:

Listing 1: Parallel pheromone deposit with OpenMP atomics

```
#pragma omp parallel for schedule(guided)
for (int ant = 0; ant < local_m; ++ant) {
    for (int e = 0; e < route_len[ant] - 1; ++e) {
        int u = route[ant][e];
        int v = route[ant][e + 1];
        double delta = deposit[ant][e];
        #pragma omp atomic
        tau[u][v] += delta;
    }
}
```

Although atomic operations introduce contention on hot edges, this design removes the dominating serial region and shifts the deposit phase back into the parallel fraction of the code.

2.2.4 Sparse delta synchronization over MPI

The most consequential redesign concerned communication. Instead of synchronizing the full dense pheromone matrix, each rank emits only the modified edges accumulated during the epoch. Communication therefore moves from a dense model to an incremental sparse-delta model based on index-increment pairs.

Listing 2: Sparse pheromone exchange through delta aggregation

```
pack_local_deltas(local_idx, local_val, &local_nnz)
;
MPI_Allgatherv(local_idx, local_nnz, MPI_INT,
               all_idx, recvcunts, displs, MPI_INT
               ,
               MPI_COMM_WORLD);
MPI_Allgatherv(local_val, local_nnz, MPI_DOUBLE,
               all_val, recvcunts, displs,
               MPI_DOUBLE,
               MPI_COMM_WORLD);
apply_remote_deltas(all_idx, all_val, total_nnz);
```

On the large instances considered in this report, this change reduces network traffic by roughly three orders of magnitude: from about 1 GB of dense state to about 2 MB of actual modifications.

2.3 Empirical validation of Version 2

The later experimental campaigns confirmed that the sparse design matches the true behavior of the solver.

2.3.1 Experiment 15: sparsity analysis

For $N = 16,000$ and $M = 1536$, the measured update density per epoch is only **0.1085%**. In ab-

solute terms, this means that on a 1 GB pheromone matrix only about **2.12 MB** of data are modified in a typical epoch. This result justifies abandoning dense collectives in favor of sparse communication.

2.3.2 Experiment 16: incremental sparse MPI synchronization

Replacing row-wise `MPI_Iallreduce` with `MPI_Allgatherv` over packed deltas produced a measured speedup of **1.61×** in a local 2-rank setup. The improvement came from eliminating the transfer of almost entirely zero-valued matrix regions. To avoid shifting the bottleneck elsewhere, pheromone normalization was also parallelized instead of being left to the master thread.

2.3.3 Experiment 17: sparse asynchronous SSP

The final Version 2 configuration combines sparse synchronization with *Stale Synchronous Parallelism* (SSP) using `MPI_Iallgatherv`. In the local shared-memory testbed the observed speedup was **1.03×**, while the expected benefit on a real multi-node cluster is estimated in the **1.2×**–**1.5×** range. The modest local gain is coherent with the extremely low latency of local MPI paths: once the network is already cheap, overlapping communication with computation removes only a small residual cost. On real clusters, however, the asynchronous design moves the network almost entirely out of the critical path.

2.4 Current status and next steps

The current solver can be summarized as an **adaptive, hierarchical, sparse-SSP** OpenMP–MPI engine. Within the scope of this project, its main properties are:

1. **Network efficiency:** approximately a **460×** reduction in MPI traffic, from dense gigabyte-scale synchronization to sparse megabyte-scale deltas.
2. **Computation–communication overlap:** useful work and collective progress proceed concurrently rather than as two strictly separated phases.
3. **High throughput on large instances:** about 770 ms per epoch at $N = 16,000$, with near-ideal local scaling in the tested environment.

The next logical steps are to validate the SSP design on 4–8 physical cluster nodes and to study a hybrid CPU–GPU extension in which the GPU accelerates score-matrix computation while the CPU retains sparse route construction and communication management.

2.5 Why intra-ant parallelism failed on CPUs

Even after the improvements above, the construction stage remained the dominant cost on very large instances, accounting for roughly 75% of the runtime at $N = 16,000$. A new Version 3 therefore explored **collaborative ant teams**: instead of mapping one ant to one thread, several threads cooperated on the same ant, with the goal of accelerating the fallback scan over distance rows of about 64 KB.

Three variants were implemented, and all of them failed.

2.5.1 V3.1: synchronous team

The first design used explicit OpenMP barriers between the leader thread, responsible for the probabilistic choice, and the worker threads, responsible for the fallback scan. With approximately 1536 ants and 16,000 construction steps per epoch, the implementation executed on the order of **24 million global barriers per epoch**. The measured epoch time rose to about **8.4 s**, compared with about **1.4 s** for the Version 2 design based on independent threads.

2.5.2 V3.2: speculative spinning

To remove barrier overhead, the second design replaced explicit barriers with generation-based spinning on shared atomic counters. In theory this avoided software barriers; in practice it created a **coherency storm**. Continuous polling on shared cache lines triggered MESI invalidations and cache-line bouncing, slowing the leader thread exactly when it needed bandwidth to read the real problem data. The execution became unstable and exceeded **5 minutes per epoch**.

2.5.3 V3.3: on-demand wakeup

The third design used `pthread_cond_wait` to park worker threads and wake them only when a heavy fallback scan was required. This eliminated busy-waiting but introduced a new bottleneck: kernel wakeup latency. A wakeup cost of about **5–10 μ s** is already comparable to, or larger than, the useful serial scan itself. Consequently, cores looked underutilized not because they had no work, but because they were repeatedly waiting on the operating system scheduler.

2.6 Technical interpretation of the failure

The failure of Version 3 is fundamentally a mismatch between task granularity and coordination cost. The fallback over a 64 KB row is a micro-granular memory task. On a modern CPU, the serial work is so short that any inter-thread pro-

TOCOL becomes proportionally too expensive. The parallel execution time can be expressed as

$$T_{par} = \frac{T_{comp}}{P} + T_{coord},$$

where T_{comp} is the serial scan time, P is the number of collaborating workers, and T_{coord} is the cost of synchronization. In our tests,

- $T_{comp} \approx 10 \mu s$,
- $T_{coord} \approx 15\text{--}20 \mu s$.

Since $T_{coord} > T_{comp}$, the collaborative version is slower than the serial one for any practical value of P . The issue is not a poor implementation of a promising idea; rather, it is a limit imposed by the physical cost of inter-core coordination on commodity multicore CPUs.

2.7 Design lesson

Taken together, these results point to a clear design principle for CPU-based ACO solvers: **one ant per thread** remains the most robust strategy for sustaining high throughput. If the goal is instead to reduce the latency of the fallback stage, the more promising path is not intra-ant collaboration across cores, but **SIMD vectorization within a single core**. Instruction sets such as AVX2 and AVX-512 can process 8–16 elements per instruction without introducing barriers, kernel wakeups, or coherency traffic, and therefore offer a more credible route to lower latency while preserving full core utilization.

3 CUDA Implementation

The GPU implementation of the ACO solver represents a fundamental paradigm shift from the sequential and hybrid baselines. To scale beyond 100,000 nodes on consumer-grade hardware, the implementation transitions from a **memory-centric** model to a **compute-centric** model, optimizing the **Memory Hierarchy** to maximize **SIMT (Single Instruction, Multiple Thread)** efficiency.

3.1 Paradigm Shift: From Memory-Centric to Compute-Centric

Traditional ACO implementations rely on a precalculated distance matrix c_{ij} to evaluate the heuristic term $\eta_{ij} = 1/c_{ij}$. While this approach minimizes arithmetic operations, it creates a catastrophic bottleneck on GPU architectures due to the high latency and finite bandwidth of Global Memory (DRAM).

3.1.1 The Memory-Bound Bottleneck

In a memory-centric model, every ant decision requires fetching a 32-bit float from Global Memory. On modern GPUs, while ALU throughput (TFLOPS) has grown exponentially, Global Memory

bandwidth (GB/s) has lagged behind. This results in a near-zero **Compute-to-Global-Memory-Access (CGMA)** ratio, capping the execution rate at a small fraction of the hardware’s theoretical peak. Furthermore, the $O(N^2)$ memory requirement creates a physical limit: for $N = 100,000$, a float matrix would require 40 GB of VRAM, exceeding the capacity of consumer hardware.

3.1.2 The Compute-Centric Advantage

The v6 architecture replaces the distance matrix with **on-the-fly Euclidean distance calculation** from an $O(N)$ array of `float2` coordinates. This transition exploits the fact that modern GPUs dedicate the vast majority of their silicon area to Energy-efficient Arithmetic Logic Units (ALUs). By replacing a high-latency DRAM lookup with a sequence of arithmetic operations (`sub`, `mul`, `add`, and hardware-accelerated `rsqrtf`), we dramatically increase the CGMA ratio, shifting the kernel from being **memory-bound** to **compute-bound**.

3.1.3 Experimental Validation

Preliminary benchmarks confirm that calculating distances on-the-fly is orders of magnitude more efficient than Global Memory lookups as N increases. The advantage already exceeds three orders of magnitude at $N = 1,000$, grows beyond four orders of magnitude by $N = 10,000$, and reaches approximately **12,500×** at $N = 30,000$. At $N = 100,000$, the lookup-based approach no longer fits in memory, while the compute-centric version remains executable.

3.2 Memory Hierarchy Strategy

The architecture adopts a tiered memory management strategy to balance extreme capacity requirements with single-cycle execution speeds.

3.2.1 Registers: Ant Operational State

The most critical optimization is keeping the entire **Ant Operational State** entirely within **Registers**. Verification via `cuobjdump` confirms that each thread utilizes exactly **48 registers**, storing 64-bit pointers, PRNG states, and candidate scores. This allocation maintains the exploration "on-chip," avoiding the performance cliff of register spilling. By limiting usage to 48 registers, the kernel achieves an **SM Occupancy of 66%** (42 active warps per SM), providing sufficient overhead for **latency hiding** during pheromone matrix fetches.

3.2.2 Exclusion of Shared Memory

A significant architectural choice is the **avoidance of Shared Memory** for intermediate calculations or route storage.

- **Capacity:** Vehicle routes for $N = 100,000$ are

too large to fit in the few dozen KB of Shared Memory per SM.

- **Synchronization:** Coordinating Shared Memory requires `__syncthreads()` barriers, which stall the entire block. Our approach uses **Warp Shuffles** for lock-free register-to-register communication.
- **Throughput:** Since on-the-fly computation frees up Global Memory bandwidth, using Shared Memory to "save" bandwidth would introduce unnecessary synchronization cycles without tangible gains.

3.2.3 Constant Memory: Invariant Parameters

Algorithmic parameters (α, β, ρ, Q) are stored in **Constant Memory**. This utilizes a dedicated on-chip cache and a **broadcast mechanism**, serving the entire warp in a single cycle and saving DRAM bandwidth for the dense pheromone updates.

3.3 Thread Mapping: Static Warper-Ant

The v6 architecture assigns exactly **one 32-thread warp to each ant**. This strategy was chosen over dynamic work queues or block-level mapping for several hardware-level reasons:

- **Divergence Control:** CVRP customer selection is stochastic. A warp-per-ant model ensures all 32 threads evaluate the same ant's candidate set simultaneously, maintaining perfect **control-flow convergence** and SIMT efficiency.
- **Memory Coalescing and Temporal Sync:** Route storage uses a **Step-Interleaved Structure of Arrays (SoA)** layout (`routes[step][ant]`). This establishes a "temporal contract" between warps: at simulation step T , all warps write their chosen nodes into contiguous addresses. A dynamic mapping would destroy this synchronization, leading to scattered, uncoalesced writes that would saturate the memory bus.

3.4 Algorithmic Innovations

- **Single-Pass Fallback:** We implement **Weighted Reservoir Sampling** to evaluate the entire graph in a single $O(N)$ pass. This resolves the ALU saturation caused by traditional two-pass roulette wheels, utilizing hardware `rsqrtf` for single-cycle distance processing.
- **AtomicCAS Workaround:** Since CUDA lacks 8-bit atomics, pheromone reinforcement is implemented via a **32-bit AtomicCAS loop** with bit-masking, ensuring race-free updates to

the log-quantized matrix.

3.5 Performance Analysis

The analysis identifies a crossover point around $N = 4000$. Below this, the CPU's cache locality prevails; above it, the GPU's exponential throughput and the compute-centric v6 architecture provide a scalable solution for ultra-large CVRP instances.

4 Performance Comparison

This section analyzes the measured scaling behavior of the implementations developed for the project relative to the sequential ACO baseline. The discussion focuses on how elapsed time changes with problem size and with the amount of parallelism, in order to identify where additional workers reduce runtime and where overhead begins to dominate. All quantitative results discussed below are derived from successful runs only. The available measurements report the problem size n , CVRP parameters K and m , solver and instance seeds, elapsed time, `best_cost`, memory footprint for CPU runs, and the parallel configuration expressed through `mpi_ranks` and `omp_threads`. They do not provide CPU or GPU model identifiers, compiler versions, or compilation flags, so the setup summary is restricted to directly observable information.

4.1 Experimental Setup

The experimental material is organized into three campaigns that capture complementary aspects of solver behavior: a baseline comparison between the sequential and CUDA versions, strong-scaling tests for the CPU backends, and weak-scaling tests in which the problem size grows with the worker count. The strong-scaling campaigns vary `omp_threads` and `mpi_ranks` at fixed n , while the weak-scaling campaign increases the problem size together with the available parallel resources. Overall, the dataset includes 15 successful baseline runs, 39 strong-scaling CPU runs, and 13 weak-scaling CPU runs over the ranges $n = 500\text{--}32000$, $n = 2000\text{--}32000$, and $n = 2000\text{--}32000$, respectively.

4.2 OpenMP Backend

The strong-scaling measurements show a clear interior optimum rather than a monotonic improvement with thread count. Up to $n = 16000$, the minimum elapsed time is obtained with four threads, reducing runtime from 115.22 s to 30.43 s at $n = 2000$ and from 165.68 s to 98.58 s at $n = 16000$. At $n = 32000$, the minimum shifts to two threads with 374.08 s, indicating that the additional parallel work exposed by larger thread counts is no longer sufficient to offset synchronization and management overhead.

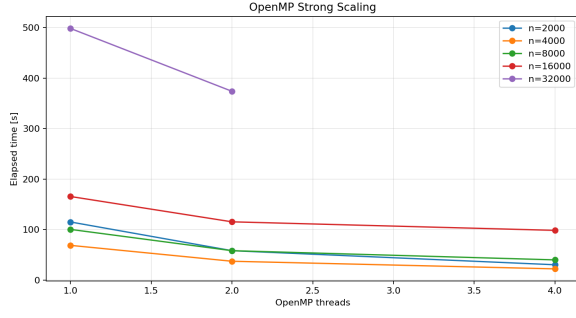


Figure 1: Strong-scaling elapsed time for the OpenMP backend. The most favorable operating region is reached with a moderate thread count.

In the weak-scaling campaign, the elapsed time remains nearly flat between the $n = 4000$ and $n = 8000$ points, with 46.16 s and 47.39 s for two and four threads respectively. Beyond that regime, the runtime rises to 89.09 s at $n = 16000$ and 268.67 s at $n = 32000$, showing that the larger working set and the growing synchronization cost already limit the extent to which additional threads preserve constant runtime at the largest remaining size.

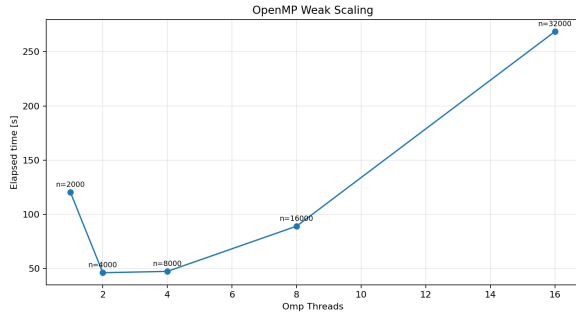


Figure 2: Weak-scaling elapsed time for the OpenMP backend. Runtime remains controlled only in the lower part of the explored size range.

The speedup curve makes the same pattern explicit. Per-worker efficiency approaches 0.95 at $n = 2000$, but the curve flattens progressively once the thread count moves beyond the moderate operating region, which indicates diminishing returns from additional parallelism.

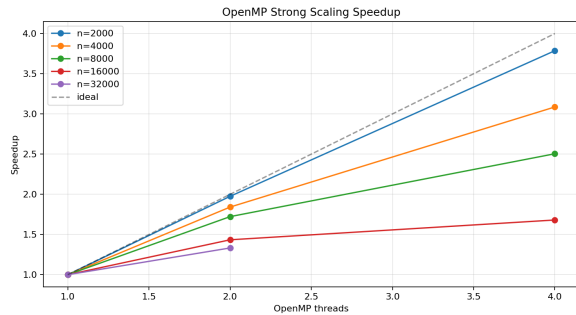


Figure 3: OpenMP speedup curve generated from the merged strong-scaling campaign. The most efficient region lies at moderate thread counts.

4.3 MPI Backend

The MPI strong-scaling data also exhibit a narrow optimal parallelism region. The minimum elapsed time is obtained with two ranks up to $n = 8000$, shifts slightly to four ranks at $n = 16000$, and returns to one rank at $n = 32000$. The change in the minimizing configuration indicates that communication overhead becomes increasingly visible as the number of ranks grows.

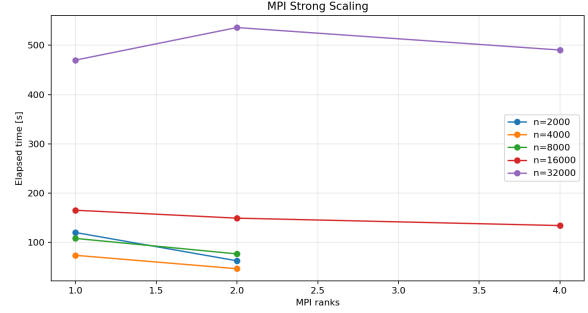


Figure 4: Strong-scaling elapsed time for the MPI backend. Performance improves at low rank counts and then saturates as communication costs accumulate.

The weak-scaling measurements remain below 60 s up to the $n = 16000$ points represented in the campaign, then climb to 129.61 s at $n = 32000$. This increase shows that the weak-scaling regime remains controlled only over the smaller part of the explored range, after which communication and coordination costs prevent near-constant execution time.

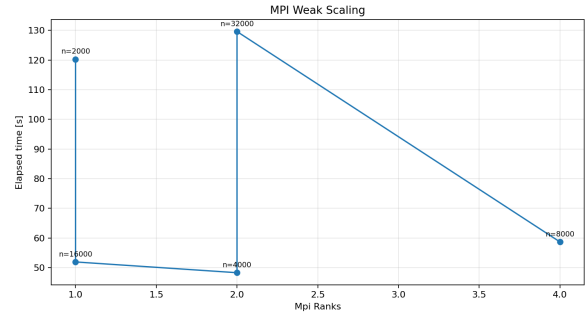


Figure 5: Weak-scaling elapsed time for the MPI backend. The cost of distributed synchronization becomes evident on the largest instances.

4.4 Hybrid OpenMP–MPI Backend

The hybrid strong-scaling measurements show that combining ranks and threads yields substantial reductions in elapsed time at smaller and medium problem sizes, reaching 12.39 s at $n = 2000$, 14.25 s at $n = 4000$, and 31.97 s at $n = 8000$. At larger sizes, however, the reduction becomes less pronounced: the minimum observed elapsed time is 122.24 s at $n = 16000$ and 453.24 s at $n = 32000$. The data therefore point to an optimal intermediate operating

region rather than sustained gains from continually increasing parallelism.

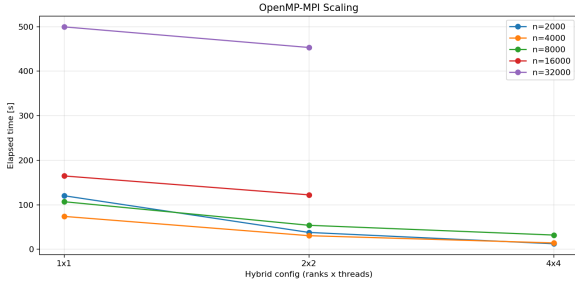


Figure 6: Strong-scaling elapsed time for the hybrid OpenMP–MPI backend. It is the strongest CPU-only configuration on the shared sizes, especially for small and medium instances.

The weak-scaling campaign is sparser, but it shows the same limitation. A 2×2 setup solves $n = 8000$ in 55.38 s, whereas 4×4 workers require 372.47 s on $n = 32000$. The joint increase of inter-rank communication and intra-rank synchronization therefore prevents the larger configurations from maintaining near-constant runtime on the largest tested cases.

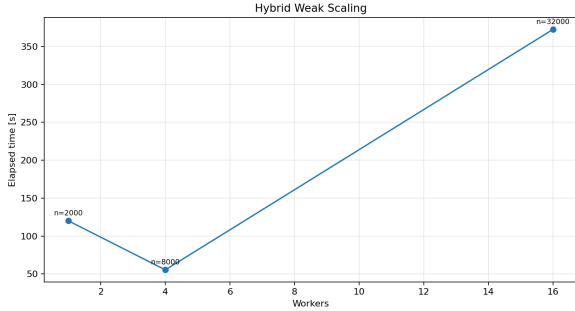


Figure 7: Weak-scaling elapsed time for the hybrid OpenMP–MPI backend. The early advantage fades once both ranks and threads increase together.

4.5 CUDA Backend

Relative to the sequential baseline, the CUDA measurements preserve a speedup larger than one at every overlapping size. The sequential-over-CUDA speedup ranges from $111.21\times$ at $n = 2000$ to $2.87\times$ at $n = 32000$, showing that the runtime gap narrows as the workload grows but remains measurable throughout the shared range.

Across the CPU-oriented strong-scaling experiments, the common pattern is that runtime reductions are concentrated in a moderate parallelism region rather than increasing monotonically with the number of workers. As the worker count grows, the minimizing configurations shift toward lower thread or rank counts, which is consistent with the increasing impact of synchronization and communication overhead. The weak-scaling results show the same limitation from a complementary perspective:

elapsed time remains comparatively stable only over the lower part of the explored range and then increases sharply once both problem size and parallel resources grow together. In the direct cross-backend comparison, the measurements further show that at $n = 32000$ all CPU parallel variants are slightly slower than the sequential baseline, whereas CUDA still retains a sequential-over-CUDA speedup above one.

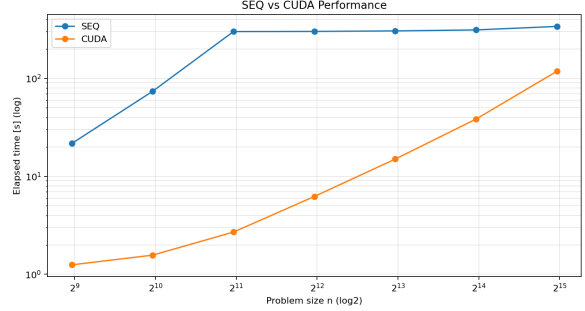


Figure 8: Direct comparison between the sequential baseline and CUDA elapsed time. CUDA dominates the overlapping problem sizes and remains usable on the largest tested instances.

5 Conclusions

The results identify a clear hierarchy among the tested implementations. CUDA is the strongest overall solution on every problem size shared with the other backends, and it remains the only approach that preserves a substantial advantage when the instance size becomes large. Among the CPU-only implementations, the hybrid OpenMP–MPI solver delivers the best absolute times on small and medium instances, whereas OpenMP alone offers the most favorable efficiency profile when the number of threads is kept moderate.

At the same time, the experiments show that raw parallelism is not sufficient to guarantee scalability. For MPI and for the hybrid backend in particular, the benefits of adding workers diminish once communication and synchronization costs become comparable to the useful computation. This effect is especially visible on the largest instances, where the coordination overhead erodes most of the performance gains observed at smaller scales.

Overall, these findings suggest that the choice of backend should depend on the target workload regime: CUDA is the preferred option whenever GPU execution is available, the hybrid solver is the most competitive CPU alternative for small-to-medium instances, and OpenMP is the most balanced shared-memory solution when simplicity and efficiency are both important.

References

- [1] Wikipedia, *Vehicle routing problem*: https://en.wikipedia.org/wiki/Vehicle_routing_problem
- [2] Wikipedia, *Ant colony optimization algorithms*: https://en.wikipedia.org/wiki/Ant_colony_optimization_algorithms